A Diffused-Interface Immersed Boundary Approach Based GPU Accelerated Incompressible Fluid-Structure Interaction Solver

A project report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

by

Palash Pai Angle

(Roll No. 230103069)

Under the supervision of

Dr. Atul Kumar Soti

Department of

Mechanical Engineering,

IIT Guwahati

April 2026-July2026



Department of Mechanical

Engineering Indian Institute of

Technology Guwahati Guwahati-

781039, Assam, India

UG/03 Form

To,
The Dean of Academic Affairs,
Indian Institute of Technology Guwahati,
Guwahati – 781 039, Assam, INDIA

Sub: Certificate in regard to completion of Internship programme by Mr. Palash Pai Angle

Sir/Madam,

This is to certify that Mr Palash Pai Angle bearing IIT Guwahati's Roll No 230103069, did his/her Summer Internship Programme in our organization, from April 2026 to July 2026. He completed Internship Programme in our organization, in the area of Application of Computational Fluid Dynamics in Flow Induced Vibrations.

His conduct and performance during the said Internship Programme was found to be:
Satisfactory

Date…….

(Name of the signatory in capital letters)
Designation of the signing authority with seal

ACKNOWLEDGMENT

I would like to offer my most sincere gratitude to Dr. Atul Kumar Soti, Assistant Professor in the Department of Mechanical Enginnering, IIT Guwahati, for his apt guidance, relentless support, and continuos motivation, providing the required counsel and direction, without which fulfilling this project work wouldn't have been possible. I would like to use this opportunity to express my appreciation for his effort.

I would also like to thank the HOD of Mechanical Engineering Department, Dr. Shyamanta Hazarika for providing me the permission to carry out this research work as a part of summer internship programme, which helped in utilizing the three-month vacation judiciously. I thank him for providing the necessary facilities required for the project work as well.

Furthermore, I'm truly grateful to my family- Dwarkanath, Sangeeta and Laximkant for their relentless support and encouragement throughout this journey. I'd like to thank my labmates too- Harsha, Kautillya, Rajdeep, Jatin, Suman, Deepak, Dhanjita and Abhishek, who often engaged in insightful discussions and intriguing arguments with me regarding the subject, which made the lab hours very productive and motivated me to take the project forward, whenever it felt difficult to do so.

Lastly, I would like to thank every single person going through this report. I hope you have a wonderful time reading it, and it enlightens your interest in the field of computational fluid dynamics, motivating you to contribute further.

Palash Pai Angle

ABSTRACT

Historically, Fluid-Structure Interaction problems have been computationally very expensive to solve, while computational cost could be minimized only at the expense of accuracy. In this work, A GPU (Graphics Processing Unit) accelerated Fluid Structure Interaction Solver is proposed. The solver aims at parallelizing every part of the code in order to make it compatible with the Single Instruction Multiple Threads (SIMT) model of GPU architecture. Three-Dimensional incompressible Navier Stokes Equations are solved using a two-step fractional step method, while solid-fluid coupling is ensured using a diffused-interface immersed boundary method. Immersed boundary method facilitates the use of simple selectively refined non body-conformal Cartesian meshes over highly unstructured meshes, thus simplifying the mesh-generation task as well as eliminating the need of re-meshing at every time step, as it is in case of moving bodies. Projection Immersed boundary approach, a variant of the Diffused interface direct forcing method, was selected since it exhibits high degree of parallelism. The subsequent linear equations resulting from the discretization are solved using a Preconditioned Conjugate Gradient Algorithm. Preconditioning is done using a Red-Black Symmetric Gauss-Siedel Algorithm, owing to its inherent parallelism. Thus the convergence is greatly accelerated by solving multiple nodes parallely using the GPU architecture, reducing the overall computational time without compromising with the accuracy. A directive-based Parallelization of the entire code has been implemented using the open-source openACC library developed by NVIDIA.

Two applications of this approach are presented. A standard lid-driven cavity problem has been solved to test the accuracy of fractional step algorithm when implemented on GPU, and further to validate the implementation of immersed boundary method, flow around a square cylinder has been simulated at various reynold numbers, demonstrating different vortex shedding patterns.

Keywords: Fluid-Structure Interaction, Immersed Boundary Methods, Parallel Computing.

TABLE OF CONTENTS

UG/03 Form **3**

ACKNOWLEDGMENT **5**

ABSTRACT **7**

INTRODUCTION **9**

1.1 MOTIVATION 9

1.2 OUTLINE OF THE REPORT AND SCOPE OF THE SOLVER 10

LITERATURE REVIEW **12**

2.1 AN OVERVIEW 12

2.2 FRACTIONAL STEP METHOD 12

2.3 KRYLOV SUBSPACE METHODS 15

2.3.1 CONJUGATE GRADIENT METHODS 16

2.3.2 PRECONDITIONING 17

2.4 IMMERSED BOUNDARY METHODS 18

2.4.1 CONTINUOUS FORCING APPROACH 19

2.4.2 DISCRETE FORCING APPROACH 20

2.4.2.1 INDIRECT BOUNDARY CONDITION IMPOSITION 20

2.4.2.2 DIRECT BOUNDARY CONDITION IMPOSITION 21

BOUNDARY MOTION 25

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

Fluid-Structure Interaction phenomena play a pivotal role in the daily lives. It has wide application in various fields, ranging from machine modeling (aircrafts, automobiles etc.) to biomedical industries, and in every case it's not advisable to move to the designing stage without having an accurate solution to the fluid flow and structure response. Hence, since past few decades, development of a generic tool for solving such multi-physics problems has been the prime focus of the research industry. The field of computational fluid dynamics has evolved a lot since late 19th and early 20th century, and a numerous computational techniques have been developed for solving various multi-physics problems, such as multiphase flows, microfluidic flows, high-reynold number turbulent flows, Rayleigh flows, high speed hypersonic flows etc., using different finite volume and finite element techniques. The main objective has always been to reduce the computational time as well as user effort without compromising with the accuracy of the solver.

Conventionally, most of the commercial solvers available in the market deploy unstructured meshes for solving fluid structure interaction problems---fig. This is due to the fact that the unstructured meshes are composed of triangular (in 2D) or tetrahedral (in 3D) elements, making it easier for them to conform around any complex geometries. This makes the application of boundary conditions on the body very straightforward, thus facilitating the simulations around complex curvilinear shapes as they are without any geometrical approximations. However, unstructured meshes come up with a lot of limitations too. Solving a problem on unstructured meshes using finite volume methods is very tedious. Since element numbering is irregular, for every element there is a need to separately define neighbouring elements and calculate flux for each, making the overall process computationally a lot more expensive compared to a structured mesh with same number of nodes. Also, since the elements here are skewed, and not orthogonal, the overall accuracy of the discretization algorithm decreases, and solution depends a lot on the mesh chosen, which creates a need of separate mesh independence study for every problem. Moreover, while simulating moving bodies, the meshing should be done for every time step, since here a body conformal mesh is required. This makes the overall process computationally very expensive and degrades the quality of the

solver.

To overcome all these limitations, an alternative approach is adopted, wherein the grids need not be conformal to the body, and the effect of the solid boundary is spread out to the nearby cells using a forcing function. Such an approach is known as Immersed Boundary Method**.**

Immersed boundary method was first introduced by Peskin in 1973 to simulate blood flow in arteries using non-body conformal Cartesian grids. Since then, a lot of variants of this method have been proposed. The objective has always been to take into account more complexity and versatility in computing fluid flows, while reducing the computational and human effort at the same time. The robustness and ease of implementation of this method in computing different flow arrangements has been proven since past few decades in various research works, namely ----

The objective of this work is to carry the research on immersed boundary methods forward and investigate its implementation procedure and efficacy in GPU environment. One similar work was carried out by --- in 2025 to simulate complex fiber-reinforced hyper-elastic cardiac models using continuous forcing approach using GPU acceleration. While this approach works well for elastic bodies, it fails drastically in the case of rigid bodies due to the absence of a suitable constitutive relation that would otherwise ensure solid-fluid coupling. Hence, in this work the focus is mainly on implementing IBM using projection approach ---, which works well for moving rigid bodies. The final objective is to use the solver for investigating various modes of flow induced vibrations around three dimensional complex curvilinear bodies at various reynold numbers.

1.2 OUTLINE OF THE REPORT AND SCOPE OF THE SOLVER

The report is divided into 5 sections as follows :

1) Introduction: This section describes the motivation and purpose behind carrying out this research work, highlighting its importance with relevance to engineering and industry. It also discusses the present stage of the research topic and the scope of the solver developed.

2) Literature Review: In this section, a thorough review on every topic pertaining to this research work has been carried out, discussing the evolution of immersed boundary methods particularly in the fluid structure interaction based problems.

3) Methodology: Here, the implementation of every method used in the solver is discussed in detail. The novel techniques have been derived explicitly while the techniques used from other references are cited.

4) Results: The performance and efficacy of the solver is discussed in this section. The physics determined by the solver is analyzed and compared with known results to validate its implementation. Further, a qualitative comparison with existing solvers is made in order to examine their relative performance.

5) Conclusion and Future Scope: The report is summarized in this section, and a few relatively important outcomes are discussed. Some future advancements that can be implemented in the solver are also mentioned.

--look once---

The solver is restricted to solving three-dimensional incompressible flows around rigid bodies. It essentially solves the Navier-Stokes equations in all three dimensions and a continuity equation to ensure mass conservation. Further, to incorporate the study of flow induced vibrations, the structural-damping equation is also solved at every time-step. To investigate shedding frequency and strouhal number, a Fast Fourier Transform (FFT) of transverse velocity (or lift coefficient) is computed at a certain location in the domain. The solver can simulate flow around moving bodies too.

CHAPTER 2
LITERATURE REVIEW

2.1 AN OVERVIEW

The field of computational fluid dynamics has been thoroughly advancing since past few decades. However, it hit a major paradigm shift in 1973, when an Indian-American scientist, Dr. Suhas Patankar, devised a new primitive variable approach for solving incompressible Navier-Stokes equation, known as SIMPLE algorithm. The introduction of SIMPLE algorithm brought a major breakthrough in CFD, since it ensured pressure-velocity coupling without introducing any explicit equation for pressure. Since then, a lot of new techniques have been devised to solve incompressible NV equations, overcoming the limitations offered by one-another. One such technique was introduced by Kim and Moin in 1984, known as fractional step method. This is a projection approach, i.e., velocity field is projected into a divergence free space, where pressure acts like a projector. Detailed discussion as well as mathematical derivation of fractional step method is discussed in section 2.2. The solver developed makes the use of pi-1 fractional step method to solve NV equations.

This chapter focuses on in-depth discussion on some of the recent and important advancement in the field of computational fluid dynamics. It starts with a discussion on fractional step method, followed by thorough mathematical explanation and derivation. Further, some of the krylov subspace linear solvers are discussed in detail. This is followed with a review of different variants of immersed boundary methods, out of which only projection step approach is discussed in detail with mathematical validation. The use of projection step is also justified in the subsequent section. The chapter ends with a superficial discussion on GPU-architecture, with a quick overview on NVIDIA's openACC library.

2.2 FRACTIONAL STEP METHOD

Consider an incompressible flow past a body as shown in Figure 1. It is governed by the following equations:

$$\nabla . \vec{u} = 0 \tag{1}$$

$$\frac{\partial \vec{u}}{\partial t} + \vec{u} . \nabla \vec{u} + \frac{1}{\rho} \nabla p - \frac{\mu}{\rho} \nabla^2 \vec{u} = 0 \quad \text{in } \Omega_f \tag{2}$$

$$with\ \vec{u} = \vec{u}_\Gamma\ on\ \Gamma_b$$

Here, $\vec{u}$ is the fluid velocity, p is the pressure, ρ is the density of the fluid and μ is its viscosity. Solid body occupies the domain Ω_b, bounded by the boundary Γ_b, and the fluid domain denoted by Ω_f.

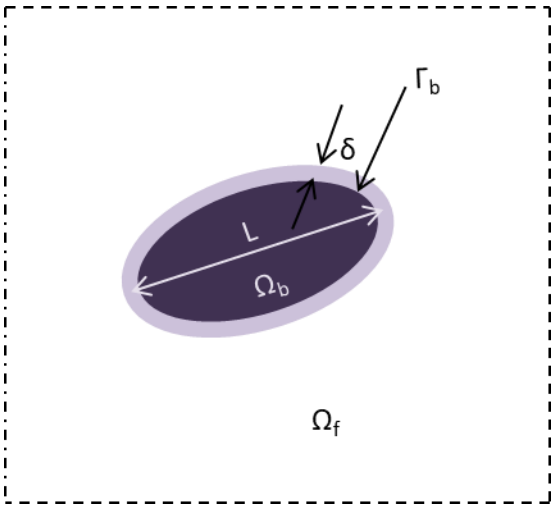


Figure 1: Fluid-Solid domain

Fractional-step method, as defined by Kim and Moin (1984), is essentially a time splitting method that relies on approximate factorization of the decomposition operators that the discretized equations contain. The role of pressure here is viewed as a projection operator that projects a random velocity field to a divergence-free vector space. The method is mainly based on the concept of Helmholtz-Hodge decomposition, according to which any arbitrary factor field can be decomposed into its divergence free and curl free counterparts.

It essentially involves 3 steps, as follows:

1) Solve Modified Momentum equations: Momentum equations are solved by completely ignoring the pressure term. Thus, following equation should be solved

$$\frac{\partial \vec{u}}{\partial t} + \vec{u} \,.\, \nabla \vec{u} - \frac{\mu}{\rho} \nabla^2 \vec{u} = 0 \qquad (1)$$

The convective terms in the current solver are discretized using second order Adams-bashforth time integration scheme, while diffusive terms are discretized using crank-Nicholson scheme.

After discretization, following equation is obtained.

$$\frac{u^*_{i,j,k} - u^n_{i,j,k}}{\Delta t} + \frac{3}{2} C^n_{i,j,k} - \frac{1}{2} C^{n-1}_{i,j,k} - \frac{1}{2}\left(D^*_{i,j,k} + D^n_{i,j,k}\right) = 0$$

Hence, an arbitrary intermediate velocity field u^* is obtained which certainly doesn't satisfy the incompressibility constraint.

2) Solve Pressure Poisson equation: A special pressure poisson equation, derived below, is solved to find the corrected pressure at that particular time step.

   Consider the original Navier Stokes equations, eq 1 and modified momentum equation, eq 2. Upon taking divergence on both sides for both the equations, and subtracting eq 1 from eq 2, following approximate equation is obtained

$$\nabla^2 p = \nabla . u^* / \Delta t$$

   Note that the cross term which remains on the rhs, has a second order error associated with it, and is hence ignored. Also, in order to ensure coupling of the pressure with velocity and prevent checker boarding, it's important to calculate divergence using face centered velocities.

3) Corrector-step: The velocities obtained from step 1 are corrected using the pressure obtained in step 2 such that the continuity constraint is satisfied. The correction step equations are as follows:

$$u_{i,j,k}^{n+1} = u_{i,j,k}^{*} - \Delta t(\frac{\partial p}{\partial x})_{i,j,k}$$

However, perot had made an interesting observation as mentioned in ---, which makes the implementation of boundary conditions as well as quantization of error much simpler and straightforward. According to it, fractional step method is essentially equivalent to carrying out block LU decomposition of the discretized momentum equations. For instance, consider the discretized version eq. 1 and 2 once again. The representation in matrix form would look somewhat like as follows

$$\begin{bmatrix} A & G \\ D & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{v^{n+1}} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{r} \\ 0 \end{bmatrix} + \begin{bmatrix} b.c.'s \\ b.c.'s \end{bmatrix}$$

Where G represents the gradient operator in discretized sub-matrix form, D is the divergence operator and $\mathbf{v^{n+1}}$ and p^{n+1} represent velocity vector and pressure at next time step, respectively. Now, the equation can be re-written as,

$$\begin{bmatrix} A & ABG \\ D & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{v^{n+1}} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{r} \\ 0 \end{bmatrix} + \begin{bmatrix} b.c.'s \\ b.c.'s \end{bmatrix}$$

where B is an approximation to A^{-1}. This can further be factored into

$$\begin{bmatrix} A & 0 \\ D & -DGB \end{bmatrix} \begin{bmatrix} \boldsymbol{v^*} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{r} \\ 0 \end{bmatrix} + \begin{bmatrix} b.c.'s \\ b.c.'s \end{bmatrix}$$

Where $\begin{bmatrix} \boldsymbol{v^*} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} I & BG \\ 0 & I \end{bmatrix} \begin{bmatrix} \boldsymbol{v^{n+1}} \\ p^{n+1} \end{bmatrix}$

For a detailed proof, ---- should be referred.

Following crank nicholson discretization scheme for diffusive terms and adams bashforth time integration scheme for advective terms, it should be noted that

$$A = \frac{1}{\Delta t}\left[I - \frac{\Delta t}{2Re}\, L \right]$$

Where I is the identity matrix of size same as that of A and L is the discrete laplacian operator. Accordingly, B can be defined as

$$B = A^{-1} = \Delta t + \frac{\Delta t^2 L}{2} + \frac{\Delta t^3 L^2}{4} + \cdots$$

$$\therefore B = \sum_{j=1}^{N} \frac{\Delta t^j}{2^{j-1}}\, L^{j-1}$$

It can be easily observed that the original fractional step method prescribed in the initial part of this section is the exact analogous of the Block LU decomposition prescribed by perot, with N=1. Here, N essentially defines the time- splitting error associated with the method. This would mean that the original fractional step is first order in time. However, this isn't the case as proven by Perot. L and G operators are commutative in nature, thus the splitting error can be easily absrorbed, preserving the second order accuracy given by discretization algorithm. For thorough derivation, kindly refer ----.

Thus, fractional step algorithm can be summarized by following three steps

1) $A\boldsymbol{u}^* = \boldsymbol{r} + bc's$
2) $G^T B G\, p = G^T \boldsymbol{u}^* + bc's$
3) $\boldsymbol{u^{n+1}} = \boldsymbol{u}^* - B\, Gp$

2.3 KRYLOV SUBSPACE METHODS

The equations resulting from discretization of the navier stokes equations need to be further solved by some efficient algorithm. The matrix resulting from discretization of NV equations is, though large, usually very sparse, or rather bended, and hence storing only non-zero diagonal suffices. This is the reason direct methods, such as LU or cholesky factorization, are very poor and computationally highly inefficient when it comes to solving of such matrices. The advantage of matrix being sparse isn't taken into account when using these methods. Hence, iterative techniques are more preferred.

Among the iterative techniques, one may mainly resort to krylov subspace methods. A krylov subspace is essentially a subspace formed by operating a matrix on a vector n-times, where n is the dimension of the vector, denoted as K_n.

$$\mathrm{K}_n(\boldsymbol{A}, \boldsymbol{x_0}) = span\{\, \boldsymbol{x_0}, \boldsymbol{Ax_0}, \boldsymbol{A^2x_0}, \boldsymbol{A^3x_0}, \ldots\ , \boldsymbol{A^{n-1}x_0}\,\}$$

These methods are essentially projection methods which seek an approximate solution x_n from an affine subspace $x_0 + \mathrm{K}_n$ by imposing petrov-Galerkin condition, i.e.

$$(\boldsymbol{b} - \boldsymbol{Ax_n}) \quad \perp \quad \Lambda_n$$

Where Λ_n is another subspace of dimension n.

It should be noted that every vector in krylov subspace can be written as $x = p(A)v, v \in R^n$. Hence, as a consequence of Cayley-Hamilton Theorem, there exists a polynomial of degree n, such that $p(A)v = 0$. Hence, it's always possible to project a vector belonging to R^n to its null-space with the use of krylov subspaces. This is why solutions for linear systems can be seeked in a krylov subspace, and is often guaranteed to be exact when calculations are carried out with infinite machine precision.

2.3.1 CONJUGATE GRADIENT METHODS

Conjugate Gradient method is a special type of krylov subspace method where $\Lambda_n = \mathrm{K}_n(A, r_0)$, r_o being the initial residual. Essentially, here the current residual at a particular iteration should be orthogonal to the entire search space. The requirement is, matrix A should be symmetric as well as positive definite. The algorithm is given as follows:

Choose an initial guess x_0. Thus, $\boldsymbol{r_0} = \boldsymbol{b} - A\boldsymbol{x_0}$

Initial search direction as $\boldsymbol{d_0} = \boldsymbol{r_0}$, and some $\epsilon > 0$.

Now, for $i = 0,1,2, \ldots\ldots\ n$, while($\|\boldsymbol{r_i}\|_2 < \epsilon$)

$$\alpha = \frac{\boldsymbol{r_i^T r_i}}{\boldsymbol{d_i^T A d_i}}$$

$$\boldsymbol{x_{i+1}} = \boldsymbol{x_i} + \alpha \boldsymbol{d_i}$$

$$\boldsymbol{r_{i+1}} = \boldsymbol{r_i} - \alpha A \boldsymbol{d_i}$$

$$\beta = \frac{\boldsymbol{r_{i+1}}^{\boldsymbol{T}} \boldsymbol{r_{i+1}}}{\boldsymbol{r_i^T r_i}}$$

$$\boldsymbol{d_{i+1}} = \boldsymbol{r_{i+1}} + \beta\, \boldsymbol{d_i}$$

End

2.3.2 PRECONDITIONING

2.4 IMMERSED BOUNDARY METHODS

Conventionally, fluid flows around complex geometries are solved using body conformal grid methods. Such an approach requires highly unstructured meshes to fit around the complex curvilinear bodies, which not only makes the process computationally very expensive, but also decreases the accuracy of the solver. Also, simulating moving boundaries would require dynamic meshing, i.e. re-meshing at every time step, which further adds on to the computational cost. Such an approach is highly undesirable, especially when the complexity of the problem to be solved increases. Hence, we adopt an alternative approach, wherein the grids need not be conformal to the body, and the effect of the solid boundary is spread out to the nearby cells using a forcing function. Such an approach is known as **Immersed Boundary Method.**

Immersed boundary method works without a body conformal mesh, for any complex curvilinear body. This gives us the flexibility to choose simple structured meshes over highly unstructured meshes, thus simplifying the grid generation task and improving the accuracy at the same time. The objective here is to include the effect of the body in the governing equations of the fluid, in terms of a forcing function. The forcing function selected should be selected such that the boundary conditions imposed by the presence of body will automatically be taken care of.

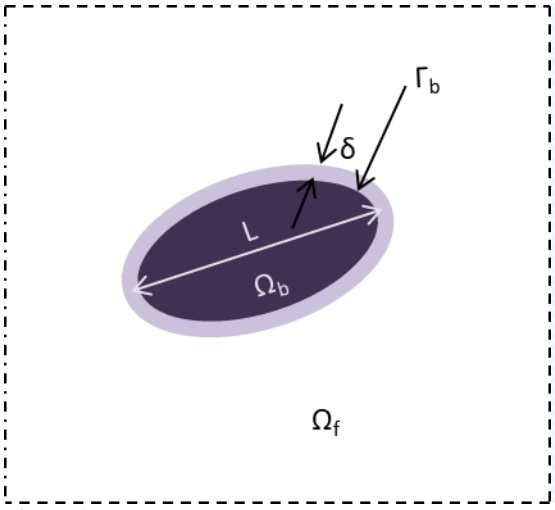


Figure 2

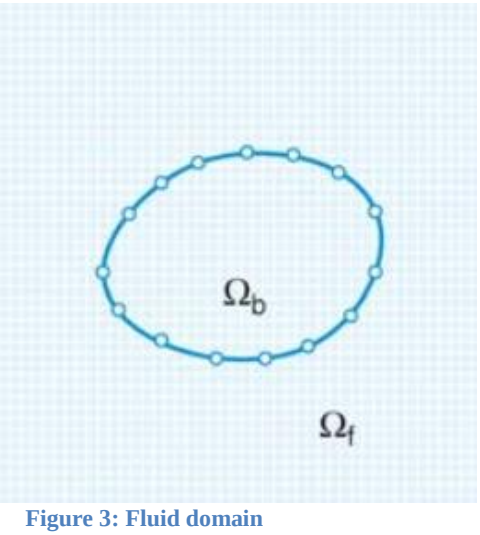


Figure 3: Fluid domain

When using a non-body Conformal mesh, application of boundary conditions becomes a difficult task. In immersed boundary method, the boundary conditions on the surface Γ_b are imposed indirectly through modifications of the governing Equations. When solving the

equation near the body boundary $\Gamma_{b,}$ the momentum equations mentioned above are modified by adding a forcing term in it, such that boundary conditions are satisfied. This forcing term mainly created the effect of a boundary immersed in the fluid, which gives the method its name. Selection of forcing term is of great importance here to ensure the accuracy of the approach.

There are two ways by which this approach can be exploited. One way is to include the forcing term as a continuous function in the governing equations, and then carry out the discretization process. This is known as continuous forcing approach, and it works very well for elastic bodies whose stiffness matrices are well defined. In this case the forcing term is defined by the constitutive relation of the solid, where solid displacement and fluid velocity are solved in a coupled manner.

In the second approach, the equations are discretized first and then the forcing function is added in a smoothed manner. This is known as discrete forcing approach. This approach gives a huge flexibility and helps in incorporating various features, as it'll be shown below. At the same time, this method is computationally very expensive to implement, especially when it comes to moving bodies.

2.4.1 CONTINUOUS FORCING APPROACH

In continuous forcing approach, the forcing term added in the Navier-Stokes equation is directly taken from the constitutive relation of the elastic solid. The elastic solid boundary is treated as a collection of the fibers, and location of these fibers is tracked in a lagrangian manner, as points X_k of every k^{th} lagrangian point. These points move with local fluid velocity, hence leading us to the equation

$$\frac{\partial \overrightarrow{X_k}}{\partial t} = \vec{u}(\overrightarrow{X_k}, t)$$

The momentum forcing term due to the solid boundary is represented with the help of dirac-delta function as follows

$$\overrightarrow{f_m}(\vec{x}, t) = \sum_k \overline{\overline{C}}\, \overrightarrow{X_k}\, \delta(|\vec{x} - \overrightarrow{X_k}|)$$

But since location of fibers (lagrangian coordinates) need not necessarily coincide with the mesh (eulerian points), the sharp delta function has to be replaced with a smoother function, denoted here by *d*, such that the effect of boundary is distributed to the various surrounding

nodes around the boundary. Thus, on a discrete mesh, at coordinates (i,j) the above relation is represented as

$$\overrightarrow{f_m}(\vec{x}_{i,j}, t) = \sum_k \overline{\overline{C}}\,\overrightarrow{X_k}\, d(|\vec{x}_{i,j} - \overrightarrow{X_k}|)$$

The choice of d depends on various factors, and is a key factor in determining the accuracy of this method. Beyer and Leveque (1992) have successfully determined the choices for smoother function that preserves the accuracy of the further spatial discretization scheme used.

The equations are solved in a coupled implicit manner using a linear solver. This method works very well with elastic bodies, but can't be implemented directly for rigid bodies, since the stiffness matrix isn't defined for them. Regardless, several authors have worked over various modifications to treat rigid body in a flow, some of which have been listed below.

2.4.2 DISCRETE FORCING APPROACH

In discrete forcing approach, the forcing term is added to the governing equations in a discrete sense around the immersed boundary, such that the boundary condition is exactly satisfied. This approach doesn't lead to any fluid solid coupling, and hence works well for any type of solid body. It's also called direct forcing approach, since forcing here is directly calculated using the boundary conditions to be applied, without involving any dynamical process. Also, this method works better for higher Reynolds number compared to continuous forcing approach. This is because here forcing term chosen can be sharp, and still an accurate solution can be obtained, unlike continuous forcing approach. This helps in capturing the thin boundary layers observed in the case of high Reynolds number flows.

However, implementation of these methods, as we'll see ahead, is computationally very expensive, compared to continuous forcing approach.

Discrete forcing approach can be majorly classified into two different types:

1. Indirect boundary conditions imposition
2. Direct boundary conditions imposition (Ghost Cell Approach)

2.4.2.1 INDIRECT BOUNDARY CONDITION IMPOSITION

As the name of this method suggests, here the boundary condition is imposed on the surface in an indirect manner. The forcing function is evaluated at every time step, such that the required

boundary conditions get imposed, and added to the governing equations. This forcing function can be formally derived for a simple analytically integrable problem, as shown by Beyer and Leveque (1992). But the Navier-Stokes equation used here isn't analytically integrable, and hence this method cannot directly be used. However, (2) developed a similar method that extracts the forcing function from the numerical solution for which an *a priori* estimate can be determined.

Following methodology was prescribed by (2):

Discretize eq.(2) as follows:

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^{n}}{\Delta t} = RHS^{n+\frac{1}{2}} + \boldsymbol{f}^{n+\frac{1}{2}}$$

Now, obtain value of forcing function $\boldsymbol{f}$, such that the desired velocity is obtained near the body cells.

$$\boldsymbol{f}^{n+\frac{1}{2}} = -RHS^{n+\frac{1}{2}} + \frac{\boldsymbol{V}^{n+1} - \boldsymbol{u}^{n}}{\Delta t}$$

Where, $\boldsymbol{V}$ is the velocity with which immersed boundary moves in the domain.

This predictor-corrector approach is used to solve for flows with body in domain. Since immersed boundary grid points may not necessarily coincide with the mesh grid points, appropriate interpolation techniques need to be used to interpolate $\boldsymbol{V}$ on the mesh grid points.

This approach is quite easy to implement and computationally cheaper. However, accuracy is largely determined by the interpolation procedure used. The regular approach used in (2) resulted in spurious velocities and mass conservation issues. This interpolation scheme was subsequently improved by Zhang and Choi, resulting in fewer computations and better accuracy even at higher Reynolds number. Conservation issues associated with all these interpolated techniques were solved by (3). Major issue was conservation of wall condition. Pressure obtained by solving equations and velocities imposed on walls weren't consistent with one another. Thus, (3) developed a scheme to obtain consistency between pressure and velocity such that the wall conditions are appropriately satisfied. With proper consideration of the interpolation procedures mentioned in (3), this method can be implemented for flows around complex geometries without much complexity.

2.4.2.2 DIRECT BOUNDARY CONDITION IMPOSITION

The forcing function derived in the previous method can be implemented in two ways: (a) schemes that spread the forcing function over the vicinity of the immersed surface and (b)

schemes that produce a local reconstruction of the solution based on the boundary values. While the former scheme was used in the previous approach, the latter scheme is used here. The boundary conditions are directly imposed on the body surface without the use of any external smoothing function (4). Thus, these methods are also known as sharp interface methods, and show good results even at High-Reynolds number flows, where higher boundary layer resolution is required. But implementing these methods is not as straightforward as previous methods, and they're computationally expensive in comparison to all the previous approaches.

GHOST CELL APPROACH

This approach attempts to obtain a higher resolution of boundaries by utilizing the ghost zone inside the body. Ghost cells are defined as solid cells that have at least one fluid neighbor, as shown in Fig 4. The value of flow variables at the ghost cell is interpolated from neighboring fluid nodes, as described below.

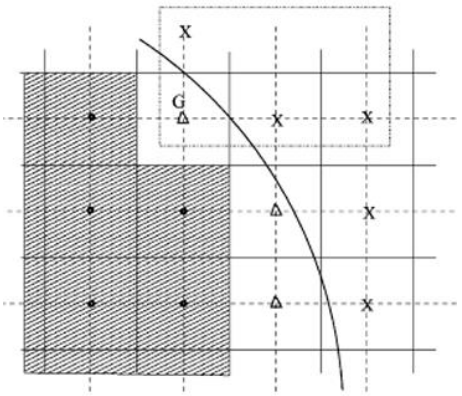


Fig 4 Ghost nodes represented by Δ and fluid nodes by x

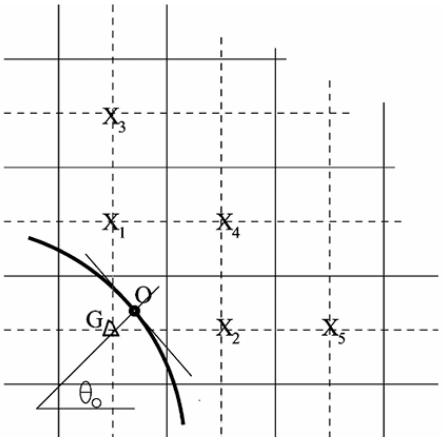


Fig 5

Originally introduced by (5), this method involved interpolation of variables at the ghost nodes using nearby fluid nodes, such that the desired boundary condition is obtained at the solid surface. The procedure as given by (5) is as follows:

1. Detect the boundary and determine the adjacent ghost cells.
2. Extrapolate to find the ghost cell value required to impose the boundary condition implicitly using an interpolation scheme.
3. Obtain the predicted field (u^*) of the fractional step procedure.
4. Solve the pressure Poisson equation to satisfy the continuity equation.
5. Update the velocity field (u^{n+1}) to the next time step.

INTERPOLATION

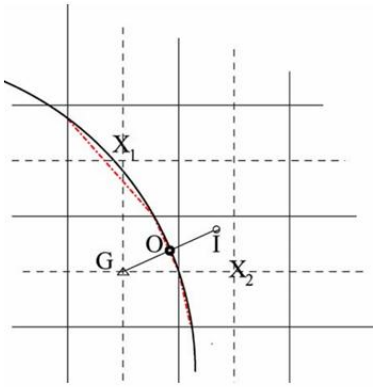


Figure 6 : Schematic of ghost cell method using image point (I)

The main task here is to interpolate the value of flow variable at the ghost node such that the boundary condition is satisfied. The easiest way to accomplish it is to draw a triangle with 2 nearest fluid nodes and one ghost node as the vertices, and then fit a linear polynomial function such that the boundary value is obtained as desired (4). With reference to the Figure 4, X_1 and X_2 are the nearest fluid nodes, and O is the vertex where boundary condition should be applied. A linear interpolation in 2D is given as

$$\varphi = a_2 x + a_1 y + a_0$$

Coefficients can be determined easily since the values of flow variables at X_1, X_2 and O is known. Thus,

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = [B]^{-1}\,[\bar{\varphi}]$$

Where, $[B] = \begin{bmatrix} 1 & x_0 & y_0 \\ 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \end{bmatrix}$, and $[\bar{\varphi}] = \begin{bmatrix} \varphi_0 \\ \varphi_1 \\ \varphi_2 \end{bmatrix}$

It's better to calculate B matrix at all the body nodes first and then carry out simulation. However, there's a major drawback associated with this approach. If ghost node location is very close to the fluid nodes used for interpolation, very high negative values of coefficients are obtained. This, although mathematically correct, leads to numerical instabilities (5). To solve this issue, we adopt a slightly different approach. Instead of interpolating ghost cell using nearest fluid nodes, an image equivalent to the ghost node is located inside the fluid domain, and interpolation is carried out only at this image node. Now since this node lies in fluid domain itself, the preceding problem won't occur. This method is shown schematically in

Figure 6.

Ghost cell value is then evaluated as: $\varphi_G = 2\varphi_O - \varphi_I$

However, implementing Neumann boundary condition using this approach is slightly cumbersome. To simplify that, (6) has slightly modified this method, wherein interpolations have been carried out only in the normal direction. In this approach, a boundary intercept point (Figure 7) is first located with respect to the ghost node, and subsequently image point is located.

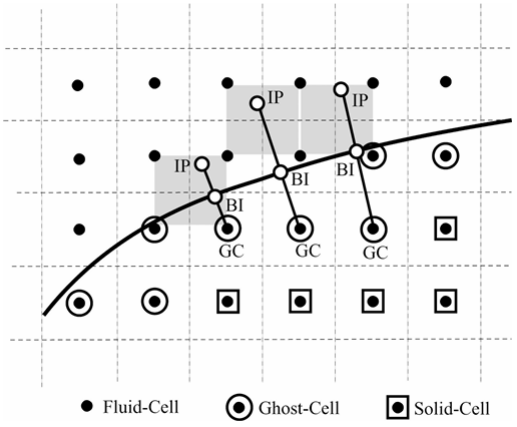


Figure 7 : Boundary Intercepts (BI) and Image Points (IP), showing ghost cell methodology

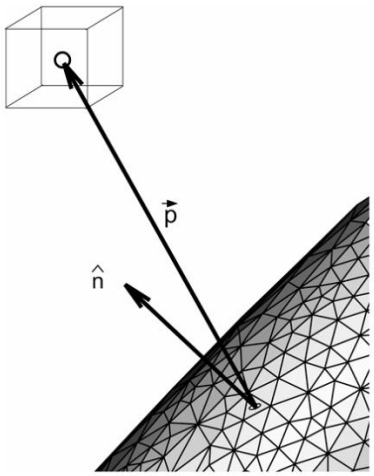


Figure 8 : Normal to the body surface, showing procedure to figure out whether the node is inside or outside the body

LOCATING SOLID CELLS

The first step here is to identify which nodes lie within a solid and which don't. The simplest

way to do so, as indicated in Figure 8, is to find the solid element nearest to the node and take dot product of surface normal of this element with the vector joining the element with node. If the dot product renders a positive value, then the node lies outside the body. Otherwise, the node is within the body (6).

IDENTIFYING BOUNDARY INTERCEPT

In principal, boundary intercept is a point on the immersed surface of the solid which is closest to the ghost cell. Ghost cells, as defined before, are solid cells which have minimum one fluid neighbor. Hence once fluid solid cells are identified, ghost cells can be subsequently located. Proper Identification of *BI* is very crucial because it is closely linked to the robustness of this method, as shown in (6). Thus, a typically complex procedure is followed for this purpose (6).
First, surface element vertex which is closest to the ghost-cell is determined. Next, the set of surface elements that share this vertex are identified and normal-intercept of these elements is searched for. In cases where multiple normal-intercepts are found, the body-intercept point is chosen to be the normal-intercept point that has the shortest intercept. For cases where no normal-intercepts are found on the surface, the search over a larger region of the surface surrounding the closest vertex is repeated. If the search is still unsuccessful, again the first set of surrounding elements are considered the point on this set of elements that is closest to the ghost-cell is searched for, keeping in mind that this closest point could even be on the edge or vertex of an element.
Once *BI* is identified, corresponding IP can be easily identified.
Ultimately, value at IP is interpolated using the values of neighboring nodes, and the ghost cell value is determined using eq. 5

BOUNDARY MOTION

Boundary motion is accomplished by advancing the lagrangian points by body velocity at (n+1) th time step, as described by the following equation.

$$\frac{X_i^{n+1} - X_i^n}{\Delta t} = V_i^{n+\frac{1}{2}}$$

The velocity can be either prescribed, or just solved if the body motion is dynamically coupled with fluid flow, as in continuous forcing approach.

Implementing boundary motion in continuous forcing approach is quite straightforward, since equations are solved in coupled manner and its quite accurate to, with no separate restrictions on time step selection. Also, its computational cost is somewhat similar to non-moving boundaries.

However, direct forcing approaches offer severe disadvantages and added complexity when it comes to implementation of moving bodies. For indirect boundary condition approach, at each time step the forcing term needs to be recomputed with a different interpolation stencil. This makes the process slightly more computationally expensive. While for sharp interface methods, entire boundary intercepts computation and ghost cell identification process needs to be repeated at every time step, making overall computation very heavy and expensive. Also, time step has to be chosen such that at any given time step, layer of fresh cells can at most be one cell layer deep (6). So the time step has to satisfy this criterion along with the CFL.